FUN GC ／ UNITY講座シリーズ

【Unity基礎】

オブジェクト指向をC#で学ぼう

第0回 UnityとC#の超入門



2026年10月15日　講師：嘉屋悠月

TODAY ・ 今日のゴール

3つできるようになろう



動かせる

キューブを矢印キーで動かせる



変えられる

動く速さを、インスペクターから変えられる



作れる

エディタ・GameObject・スクリプトを使って、1本のスクリプトを作れる

POINT　用語は後から。まず動かして体験してから、名前を付けよう。

TODAY・90分

今日の流れ

01	エディタの見方	5分
02	GameObject と Component	5分
03	初めてのスクリプト	7分
04	C#の最低限の文法（関数を含む）	48分
05	演習：キューブを動かす	20分
06	まとめ・次回予告	5分

CHECK

始める前に：準備チェック

- [ ] Unity Hub をインストールした
- [ ] Unity 6 をインストールした
- [ ] VS Code か Visual Studio を入れた
- [ ] 新しい 3D プロジェクトを作って開けた

補完が効くか確認

コードエディタで Debug. と打って、候補の一覧が出ればOK。出ないとタイプミスで詰まりやすい。

まだの人は手を挙げて

インストールは時間がかかるので、今日は隣の人の画面を一緒に見ながら進めよう。

SECTION

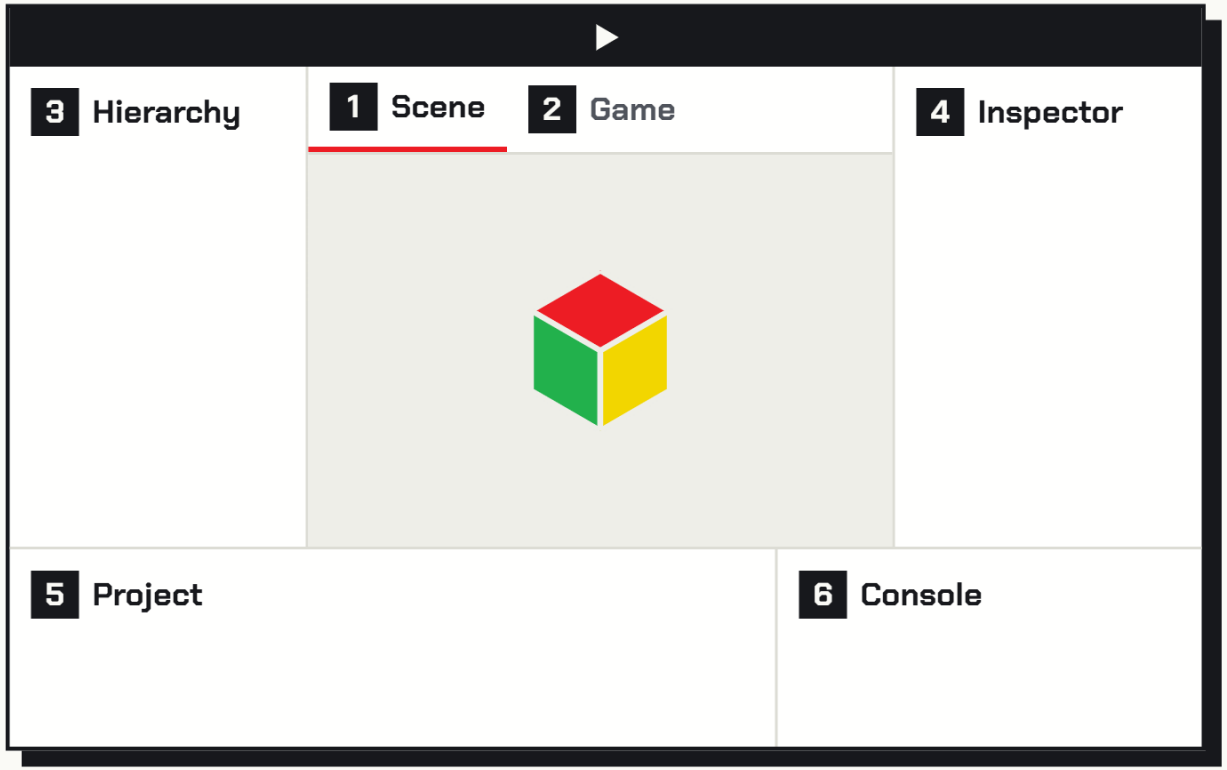
01

エディタの見方

Unityの画面にある6つのウィンドウと、
シーンの見回し方をおぼえよう



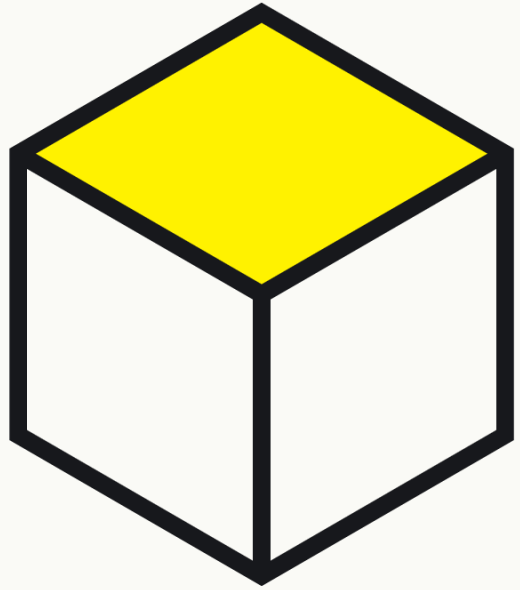
覚えるウィンドウは6つだけ



1 Scene	ゲームの世界を作る・編集する場所
2 Game	実際のプレイ画面（再生すると動く）
3 Hierarchy	シーンに置いてあるモノの一覧
4 Inspector	選んだモノの詳しい設定
5 Project	素材やスクリプトのファイル置き場
6 Console	ログとエラーが表示される場所

POINT ここだけは覚える

再生中の変更は、停止すると元に戻る。



いい数値が見つかったらメモしてから停止しよう。

SECTION

02

GameObject と Component



Unityのモノは「箱」と「部品」でできている

GameObjectは「箱」、Componentは「機能」

GameObject

シーンに置くすべてのモノ。
それだけでは何もしない「空の箱」。

Component

箱に付ける部品。
付けた分だけ、できることが増える。

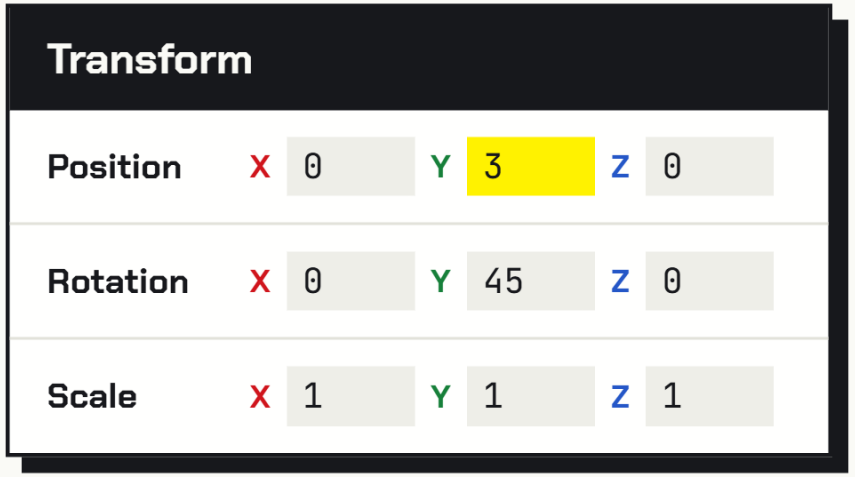
たとえるなら

スマホ本体（箱）にアプリ（部品）を
入れると、できることが増えるのと同じ。



やってみよう：キューブに機能を足す

1. Hierarchy で右クリック →「3D Object」→「Cube」
2. Inspector の Transform で、Position・Rotation・Scale の数字を変えてみる
3. 「Add Component」から Rigidbody を追加
4. **再生ボタンを押す → キューブが落ちる！**



Position の Y を 3 にすると、空中から落ちるのがよく見える

SECTION

03

初めてのスクリプト

自分で書いたプログラムを、部品としてキューブに付けよう



スクリプトを作って、キューブに付ける

1. Project で右クリック →「Create」→「Scripting」→「MonoBehaviour Script」
2. **名前を Hello にする**（作ったその場で決める）
3. Hello を、Hierarchy の Cube にドラッグ＆ドロップ
4. Hello をダブルクリック → コードエディタが開く

⚠ 名前はあとから変えない

ファイル名とクラス名は同じでないといけない。あとからファイル名だけ変えると、ずれてアタッチできなくなる。

間違えたら、消して作り直すのが一番早い。

中身を書いて、Consoleに表示しよう

Hello.cs

```
using UnityEngine;

public class Hello : MonoBehaviour
{
    void Start()
    {
        Debug.Log("Hello, Unity!");
    }

    void Update()
    {
        Debug.Log("Update!");
    }
}
```

① クラス名 ＝ ファイル名

Hello.cs なら class Hello。
自動で書かれているので触らない。

② Debug.Log( )

( ) の中身を Console に表示する。
文字は " " でかこむ。

③ 行の最後に ;

命令の終わりの合図。
忘れるとエラーになる。

Start と Update の違い

void Start()

ゲーム開始時に 1回だけ

```
Hello, Unity!
```

使い道：最初の準備（初期設定）

void Update()

毎フレーム（1秒に数十回）

```
Update!
Update!
Update!
Update!
……
```

999+

使い道：動き・キー入力のチェック

ゲームはパラパラ漫画。1コマ（1フレーム）ごとに Update が呼ばれる。

SECTION

04

C#の最低限の文法

変数・if・関数。演習に必要な3つだけをおさえよう



変数：値を入れておく「名前つきの箱」

```
int hp = 100;
```

型（種類）　名前　入れる値

「hp という名前の整数の箱を作って、100 を入れる」と読む

型	入るもの	書き方の例
int	整数（1, 100, -5）	int hp = 100;
float	小数（0.5, 3.14）	float speed = 5.5f;
string	文字列	string name = "Slime";
bool	true か false	bool isAlive = true;

よくあるつまずき：小数の「f」

⚠ NG　エラーになる

```
float speed = 5.5;
```

何も付けない小数は double 型。
float の変数には入らない。

✓ OK　正しく動く

```
float speed = 5.5f;
```

末尾に f を付けると
float 型の数として扱われる。

エラーが出たら、まず Console の赤い文字を読もう。

if 文：「もし〜なら」で分ける

Hello.cs ／ Start() の中

```
int hp = 0;

if (hp ⩽ 0)
{
    Debug.Log("ゲームオーバー");
}
else
{
    Debug.Log("まだ戦える！");
}
```

① ( ) の中が条件

「hp が 0 以下なら」

② 当てはまれば { } の中

当てはまらなければ else 側へ

③ 比べる記号

＝	等しい	≠	等しくない
<	より小さい	>	より大きい
⩽	以下	⩾	以上

関数：処理をまとめて名前を付ける

Hello.cs ／ class の中

```
void Start()
{
    SayHello();
    SayHello();
}

void SayHello()
{
    Debug.Log("こんにちは！");
}
```

① 作る

void 名前() { 中身 } で、
処理のまとまりに名前を付ける。

② 呼ぶ

名前(); と書くと中身が動く。
何回でも使える（今回は2回表示）。

③ 実は Start も関数

Start や Update は、Unity が
自動で呼んでくれる特別な関数。

補足 ・ まず動かそう

同じ関数なのに、結果が変わる

Hello.cs ／ class の中

```
void Start()
{
    Greet("スライム");
    Greet("勇者");
}

void Greet(string name)
{
    Debug.Log(name + "、こんにちは！");
}
```

Greet("スライム") の結果

スライム、こんにちは！

Greet("勇者") の結果

勇者、こんにちは！

？ どちらも Greet を呼んだだけ。かっこの中に書いたものは、どこへ行った？

かっこの中は、関数に渡す「材料」

引数（ひきすう）

関数に渡す材料のこと。
「型 名前」で書く(変数の宣言と同じ)。

呼ぶとき

Greet("勇者") と書くと、
"勇者" が name の箱に入る。

材料はいくつでも

Damage(30, 12) のように、
カンマで区切って複数渡せる。

Greet	関数
Greet	関数の名前
string	材料の型（種類）
name	材料を入れる箱の名前
Greet("勇者")	**"勇者" が name に入る**

補足 ・ もう一つ動かそう

関数に計算してもらって、答えをもらう

Hello.cs ／ class の中

```
void Start()
{
    int answer = Add(3, 4);
    Debug.Log(answer);
}

int Add(int a, int b)
{
    return a + b;
}
```

Console に出た数

7

answer の箱の中身

7

? **Add(3, 4) を呼んだら、answer に 7 が入った。7 はどうやって Start に戻ってきた？**

return は、結果を呼んだ場所へ「返す」

戻り値（もどりち）

関数が返す結果のこと。
return の後ろの値が、呼んだ場所に戻る。

返す型を先頭に書く

int Add なら「整数を返す」関数。
返さない関数は void(SayHello と同じ)。

変数の箱で受け取る

answer ＝ Add(3, 4); のように、
返ってきた結果を箱に入れて使う。




```
int answer = Add(3, 4);
```

7 が返ってきて、answer の箱に入る

材料を渡して、結果を受け取る

Hello.cs ／ class の中

```
void Start()
{
    int dmg = CalcDamage(30, 12);
    Debug.Log(dmg); // 18
}

int CalcDamage(int attack, int defense)
{
    int result = attack - defense;
    return result;
}
```

① 引数はカンマで区切る

attack と defense の2つ。
呼ぶときも同じ順番で渡す。

② return で結果を返す

計算した result が、
呼んだ場所に戻る。

③ 変数で受け取る

dmg の箱に 18 が入る。
あとで何度でも使える。

よくあるつまずき：return の書き忘れ

⚠ NG　エラーになる

```
int Add(int a, int b)
{
    int sum = a + b;
}
```

int と書いたのに、返す値がない。
CS0161: not all code paths return a value

✓ OK　正しく動く

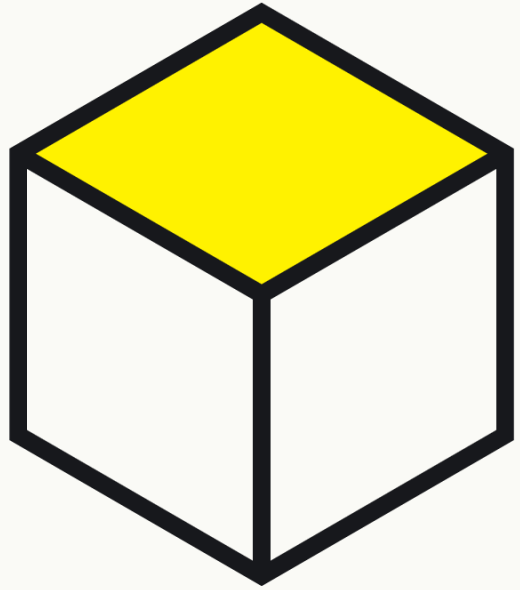
```
int Add(int a, int b)
{
    int sum = a + b;
    return sum;
}
```

return で値を返すと、
int の約束を守れる。

int と書いたら、必ず return で値を返そう。

POINT ここだけは覚える

材料（引数）を渡して、結果（戻り値）を返す。



結果を返さない関数は void と書こう。

EXERCISE

演習

目安時間

8分

補足・演習

回復する関数を作ろう

ゴール　Heal(50, 30) を呼ぶと、
Console に 80 と表示される

1. int Heal(int hp, int amount) を書く
2. 中で hp ＋ amount を return する
3. Start() で int result ＝ Heal(50, 30); と呼ぶ
4. Debug.Log(result); で 80 を確認する

早く終わった人へ
100 を超えたら 100 にする(if を使う) ／ 数を変えて何回か呼んでみる

補足・解答例

回復する関数 Heal

Hello.cs ／ class の中

```
void Start()
{
    int result = Heal(50, 30);
    Debug.Log(result); // 80
}

int Heal(int hp, int amount)
{
    return hp + amount;
}
```

① 引数は2つ

hp(いまの体力)と amount(回復量)を受け取る。

② 足した結果を返す

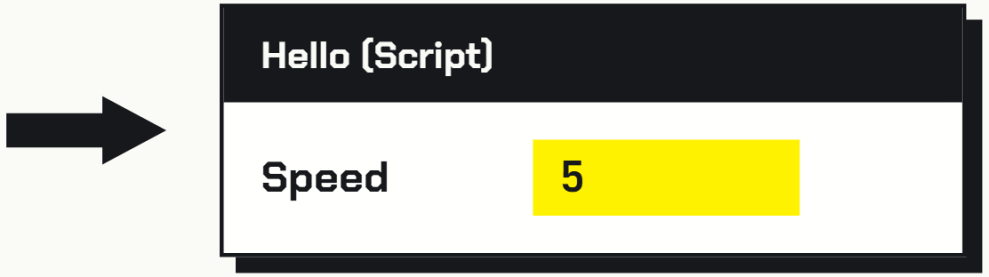
return hp ＋ amount; で 80 が戻る。

③ 変数で受け取る

result の箱に 80 が入り、Console に出る。

public を付けると Inspector に出る

```
public float speed = 5f;
```



コードを書き換えずに調整できる

再生中に数字を変えると、すぐ動きに反映される。
（ただし停止すると元に戻る！）

public の本当の意味は？

実は「外から見える・さわれる」という意味。
くわしくは第2回「カプセル化」で！

演習の前に

Unity 6 の注意：キー入力の設定

Unity 6 は新しい入力の仕組みが標準。ネットでよく見る **Input.GetKey()** は、そのままだとエラーになる。今日は設定を「Both」にして使えるようにする。

1. Edit →「Project Settings」
2. 「Player」→「Other Settings」を開く
3. **「Active Input Handling」を「Both」に**
4. 再起動を求められたら「Apply」

⚠ Console

```
InvalidOperationException: You are
trying to read Input using the
UnityEngine.Input class, but you have
switched active Input handling to Input
System package in Player Settings.
```

この赤いエラーが出たら、左の設定を見直そう

EXERCISE

演習

目安時間

20分

キューブを矢印キーで動かそう

ゴール　矢印キーでキューブが動き、
その速さをインスペクターから変えられる

1. Mover.cs を作って Cube にアタッチ
2. public float speed = 5f; を書く
3. Update() で、if と Input.GetKey で矢印キーを調べる
4. 押されていたら transform.position を動かす

早く終わった人へ

スペースキーでジャンプさせる ／ 画面の外に出たら Console に表示する

解答例：Mover.cs

Mover.cs

```
public class Mover : MonoBehaviour
{
    public float speed = 5f;
    void Update()
    {
        float x = 0f;
        float z = 0f;
        if (Input.GetKey(KeyCode.RightArrow)) x = 1f;
        if (Input.GetKey(KeyCode.LeftArrow)) x = -1f;
        if (Input.GetKey(KeyCode.UpArrow)) z = 1f;
        if (Input.GetKey(KeyCode.DownArrow)) z = -1f;
        Vector3 move = new Vector3(x, 0f, z);
        transform.position += move * speed * Time.deltaTime;
    }
}
```

① Input.GetKey( )

そのキーが押されている間だけ true になる。

② Vector3(x, y, z)

3つの数をまとめた「向き」。y は上下なので今回は 0。

③ Time.deltaTime

前のフレームからの時間。かけると PC の速さに関係なく同じ速さで動く。

今日できるようになったこと



エディタ

6つのウィンドウの役割と、シーンの見回し方。再生中の変更は戻る！



GameObject × Component

箱に部品を付けて、機能を足していく



スクリプト

Start は1回、Update は毎フレーム。変数・if・関数で動きを書く

NEXT **第1回　クラスとインスタンス**

今日書いたスクリプトの正体は「クラス」。次回はその話をします。